

Diabetes Prediction Using Machine Learning

```
#File impored
from google.colab import files
uploaded=files.upload()
```

Choose files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
#All libraries imported
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df= pd.read_csv('diabetes_dataset.csv')
```

```
#Check the uppermost to column to confirm file is imported or not
df.head()
```

	age	gender	ethnicity	education_level	income_level	employment_status	smoking_status	alcohol_consumption_per_week	physical_activity_minutes_per_week
0	58	Male	Asian	Highschool	Lower-Middle	Employed	Never		0
1	48	Female	White	Highschool	Middle	Employed	Former		1
2	60	Male	Hispanic	Highschool	Middle	Unemployed	Never		1
3	74	Female	Black	Highschool	Low	Retired	Never		0
4	46	Male	White	Graduate	Middle	Retired	Never		1

5 rows × 31 columns

```
#To know no.of rows and columns in dataset
df.shape
```

(100000, 31)

```
#To know column names
df.columns
```

```
Index(['age', 'gender', 'ethnicity', 'education_level', 'income_level',
       'employment_status', 'smoking_status', 'alcohol_consumption_per_week',
       'physical_activity_minutes_per_week', 'diet_score',
       'sleep_hours_per_day', 'screen_time_hours_per_day',
       'family_history_diabetes', 'hypertension_history',
       'cardiovascular_history', 'bmi', 'waist_to_hip_ratio', 'systolic_bp',
       'diastolic_bp', 'heart_rate', 'cholesterol_total', 'hdl_cholesterol',
       'ldl_cholesterol', 'triglycerides', 'glucose_fasting',
       'glucose_postprandial', 'insulin_level', 'hba1c', 'diabetes_risk_score',
       'diabetes_stage', 'diagnosed_diabetes'],
      dtype='object')
```

```
#To know Total rows, Non-null values and Data types
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                   100000 non-null  int64
1   gender                               100000 non-null  object
2   ethnicity                             100000 non-null  object
3   education_level                       100000 non-null  object
4   income_level                           100000 non-null  object
5   employment_status                     100000 non-null  object
6   smoking_status                         100000 non-null  object
7   alcohol_consumption_per_week           100000 non-null  int64
8   physical_activity_minutes_per_week      100000 non-null  int64
9   diet_score                             100000 non-null  float64
10  sleep_hours_per_day                     100000 non-null  float64
11  screen_time_hours_per_day               100000 non-null  float64
12  family_history_diabetes                 100000 non-null  int64
```

```
13 hypertension_history      100000 non-null int64
14 cardiovascular_history    100000 non-null int64
15 bmi                        100000 non-null float64
16 waist_to_hip_ratio        100000 non-null float64
17 systolic_bp               100000 non-null int64
18 diastolic_bp              100000 non-null int64
19 heart_rate                 100000 non-null int64
20 cholesterol_total         100000 non-null int64
21 hdl_cholesterol           100000 non-null int64
22 ldl_cholesterol           100000 non-null int64
23 triglycerides              100000 non-null int64
24 glucose_fasting            100000 non-null int64
25 glucose_postprandial       100000 non-null int64
26 insulin_level              100000 non-null float64
27 hba1c                      100000 non-null float64
28 diabetes_risk_score        100000 non-null float64
29 diabetes_stage             100000 non-null object
30 diagnosed_diabetes         100000 non-null int64
dtypes: float64(8), int64(16), object(7)
memory usage: 23.7+ MB
```

```
#checked any null value is present in any column or not
df.isnull().sum()
```

	0
age	0
gender	0
ethnicity	0
education_level	0
income_level	0
employment_status	0
smoking_status	0
alcohol_consumption_per_week	0
physical_activity_minutes_per_week	0
diet_score	0
sleep_hours_per_day	0
screen_time_hours_per_day	0
family_history_diabetes	0
hypertension_history	0
cardiovascular_history	0
bmi	0
waist_to_hip_ratio	0
systolic_bp	0
diastolic_bp	0
heart_rate	0
cholesterol_total	0
hdl_cholesterol	0
ldl_cholesterol	0
triglycerides	0
glucose_fasting	0
glucose_postprandial	0
insulin_level	0
hba1c	0
diabetes_risk_score	0
diabetes_stage	0
diagnosed_diabetes	0

dtype: int64

```
#checked any dupliacte value is present in any column or not
df.duplicated().sum()
```

```
np.int64(0)
```

```
#Checked Target Distribution since target variable is diagnosed_diabetes
df['diagnosed_diabetes'].value_counts()
```

	count
diagnosed_diabetes	
1	59998
0	40002

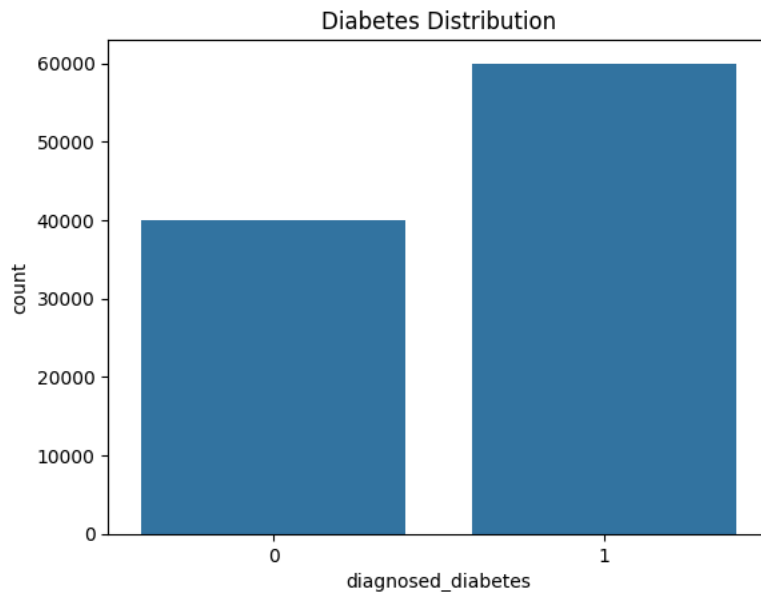
dtype: int64

```
#Checked Target Distribution in percentage since target variable is diagnosed_diabetes
df['diagnosed_diabetes'].value_counts(normalize=True)*100
```

diagnosed_diabetes	proportion
1	59.998
0	40.002

dtype: float64

```
#Plot Target distribution
sns.countplot(x='diagnosed_diabetes', data=df)
plt.title("Diabetes Distribution")
plt.show()
```



```
#Finding Numerical & Categorical Columns
num_cols = df.select_dtypes(include=['int64', 'float64']).columns

cat_cols = df.select_dtypes(include='object').columns

print("Numerical Columns:")
print(num_cols)

print("\nCategorical Columns:")
print(cat_cols)
```

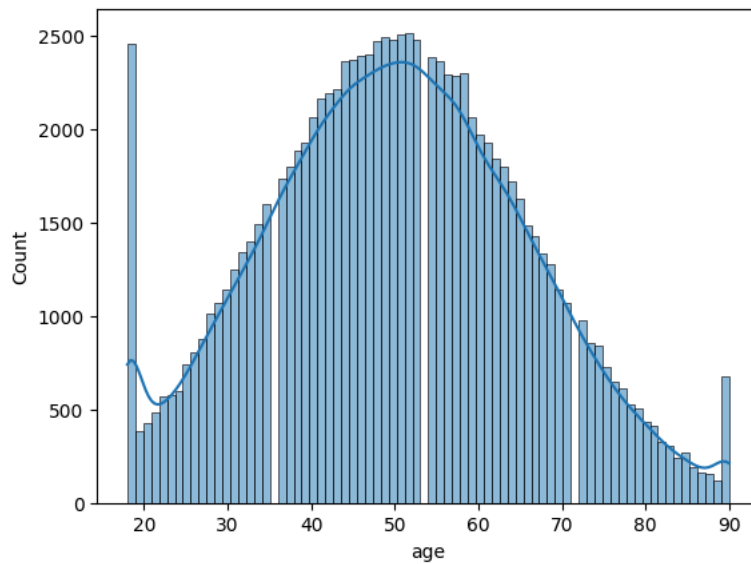
```
Numerical Columns:
Index(['age', 'alcohol_consumption_per_week',
       'physical_activity_minutes_per_week', 'diet_score',
       'sleep_hours_per_day', 'screen_time_hours_per_day',
       'family_history_diabetes', 'hypertension_history',
       'cardiovascular_history', 'bmi', 'waist_to_hip_ratio', 'systolic_bp',
       'diastolic_bp', 'heart_rate', 'cholesterol_total', 'hdl_cholesterol',
       'ldl_cholesterol', 'triglycerides', 'glucose_fasting',
       'glucose_postprandial', 'insulin_level', 'hba1c', 'diabetes_risk_score',
       'diagnosed_diabetes'],
      dtype='object')

Categorical Columns:
Index(['gender', 'ethnicity', 'education_level', 'income_level',
       'employment_status', 'smoking_status', 'diabetes_stage'],
      dtype='object')
```

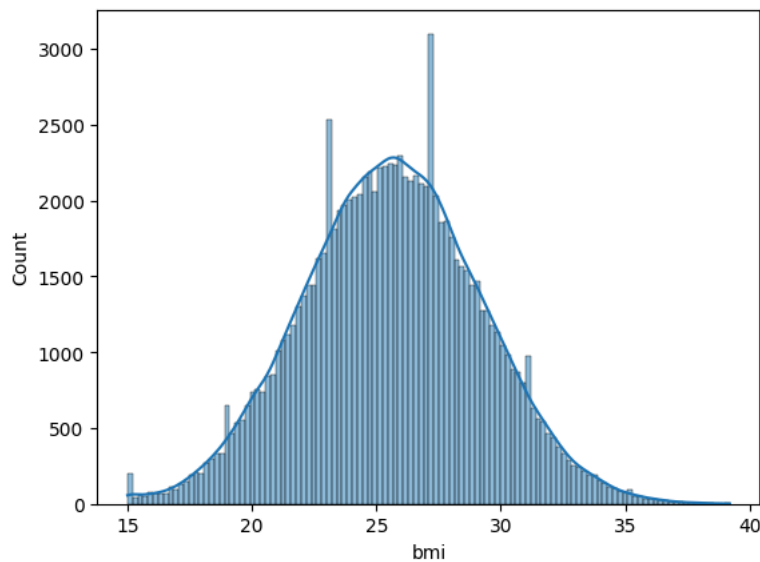
✦ Not focusing on all 30 columns just focused on important columns

```
#Age
sns.histplot(df['age'], kde=True)
```

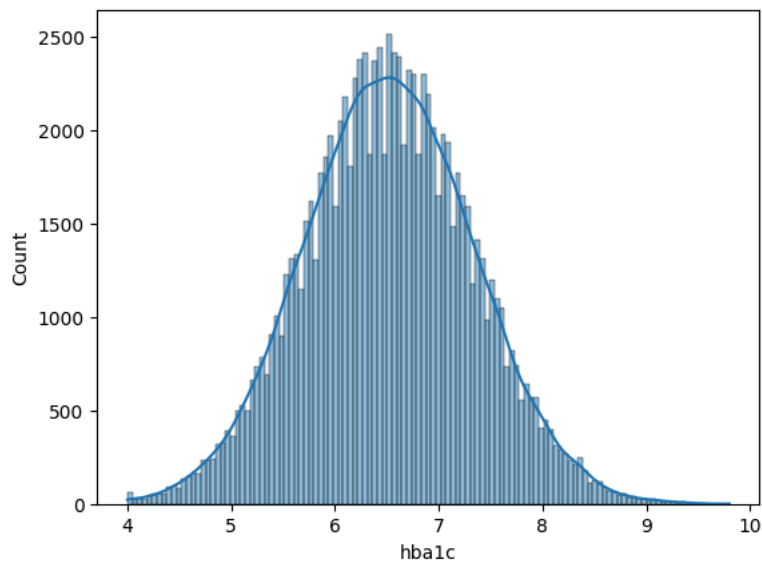
```
plt.show()
```



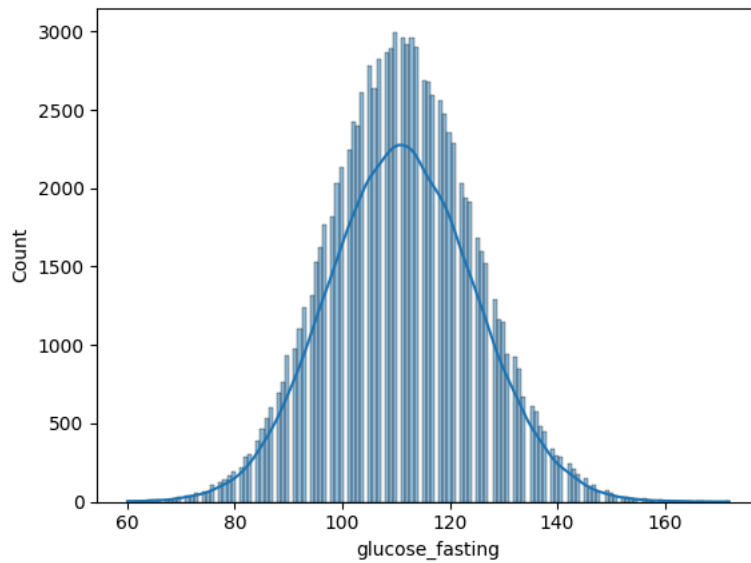
```
#BMI  
sns.histplot(df['bmi'], kde=True)  
plt.show()
```



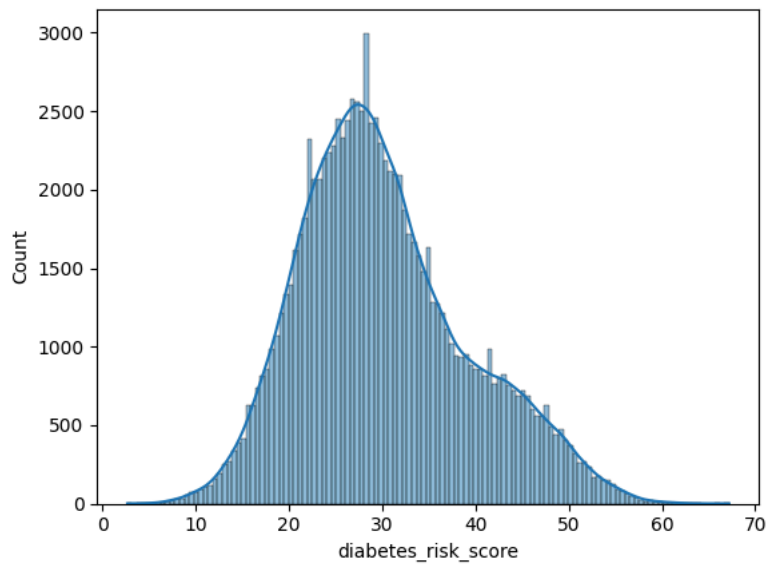
```
#HbA1c  
sns.histplot(df['hba1c'], kde=True)  
plt.show()
```



```
#glucose_fasting
sns.histplot(df['glucose_fasting'], kde=True)
plt.show()
```

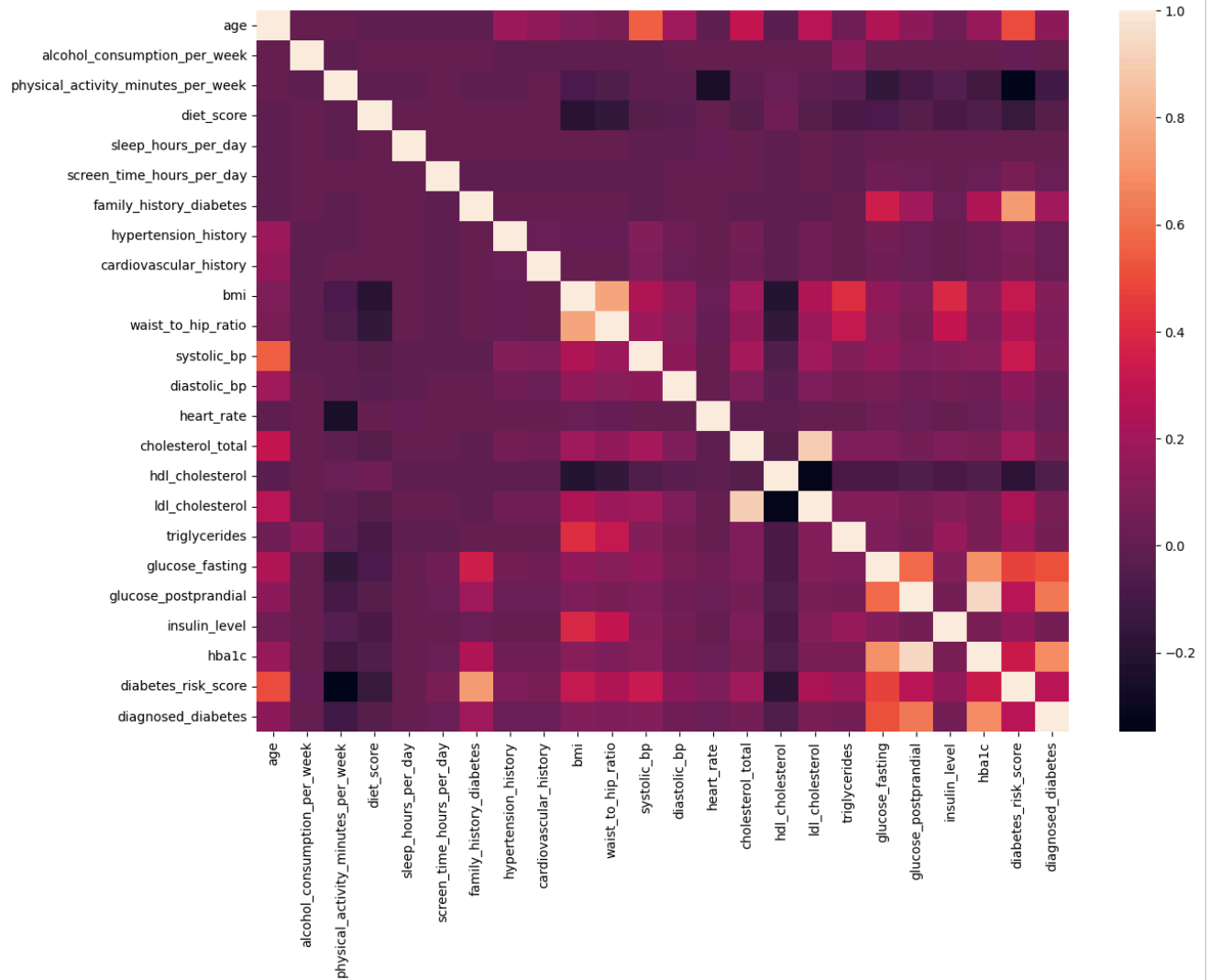


```
#diabetes_risk_score
sns.histplot(df['diabetes_risk_score'], kde=True)
plt.show()
```



Correlation Heatmap

```
plt.figure(figsize=(14,10))
corr = df.select_dtypes(include=['int64','float64']).corr()
sns.heatmap(corr)
plt.show()
```



Feature Selection

```
df[['diabetes_stage', 'diagnosed_diabetes']].head()
```

	diabetes_stage	diagnosed_diabetes
0	Type 2	1
1	No Diabetes	0
2	Type 2	1
3	Type 2	1
4	Type 2	1

```
pd.crosstab(df['diabetes_stage'], df['diagnosed_diabetes'])
```

diagnosed_diabetes	0	1
diabetes_stage		
Gestational	120	158
No Diabetes	7981	0
Pre-Diabetes	31845	0
Type 1	56	66
Type 2	0	59774

Encoding

```
pd.get_dummies(df)
```

	age	alcohol_consumption_per_week	physical_activity_minutes_per_week	diet_score	sleep_hours_per_day	screen_time_hours
0	58	0	215	5.7	7.9	
1	48	1	143	6.7	6.5	
2	60	1	57	6.4	10.0	
3	74	0	49	3.4	6.6	
4	46	1	109	7.2	7.4	
...	...	...	...	...	...	...
99995	46	1	136	8.3	6.4	
99996	41	3	76	8.8	6.8	
99997	57	4	121	9.9	5.0	
99998	47	3	52	5.9	6.7	
99999	52	1	248	2.7	5.4	

100000 rows × 53 columns

Train-Test Split

```
X = df.drop('diagnosed_diabetes', axis=1)
y = df['diagnosed_diabetes']
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

```
pd.crosstab(
    df['diabetes_stage'],
    df['diagnosed_diabetes']
)
```

diagnosed_diabetes	0	1
diabetes_stage		
Gestational	120	158
No Diabetes	7981	0
Pre-Diabetes	31845	0
Type 1	56	66
Type 2	0	59774

```
#Remove Leakage Column
df = df.drop('diabetes_stage', axis=1)
```

```
#Prepare Features and Target
X = df.drop('diagnosed_diabetes', axis=1)
y = df['diagnosed_diabetes']
```

```
#Encode Categorical Columns
cat_cols = X.select_dtypes(include='object').columns
print(cat_cols)
```

```
Index(['gender', 'ethnicity', 'education_level', 'income_level',
      'employment_status', 'smoking_status'],
      dtype='object')
```

```
#Encoding
X = pd.get_dummies(X, drop_first=True)
```

```
X.shape
```

```
(100000, 41)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

```
X_train.shape
X_test.shape
```

```
(20000, 41)
```

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
    LogisticRegression
```

```
LogisticRegression(max_iter=1000)
```

```
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.83975
```

```
print(confusion_matrix(y_test, y_pred))
```

```
[[ 6245 1832]
 [ 1373 10550]]
```

```
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

     0       0.82       0.77       0.80       8077
     1       0.85       0.88       0.87      11923

 accuracy          0.84          0.84          0.84      20000
  macro avg       0.84          0.83          0.83      20000
 weighted avg     0.84          0.84          0.84      20000
```

```
print("Train Score:", model.score(X_train, y_train))

print("Test Score:", model.score(X_test, y_test))
```

```
Train Score: 0.843925
Test Score: 0.83975
```

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier()

knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_knn))
```

```
Accuracy: 0.80845
```

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_dt))
```

```
Accuracy: 0.8623
```

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Accuracy: 0.9202
```

```
from sklearn.metrics import classification_report

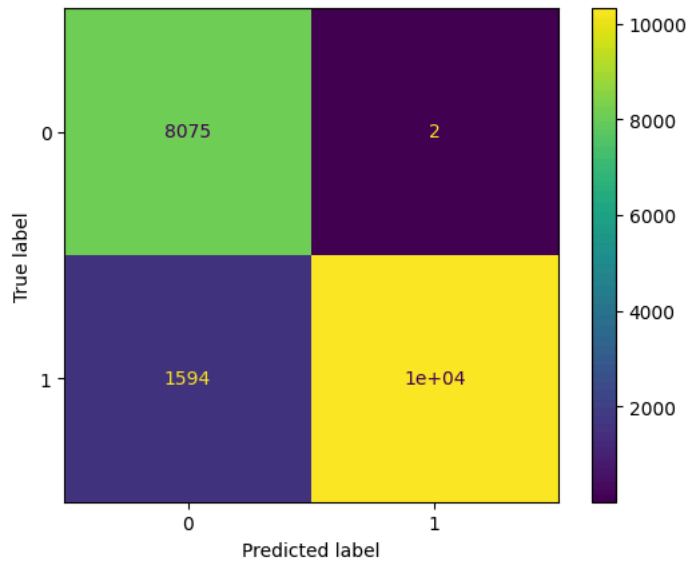
y_pred_rf = rf.predict(X_test)

print(classification_report(y_test, y_pred_rf))
```

	precision	recall	f1-score	support
0	0.84	1.00	0.91	8077
1	1.00	0.87	0.93	11923
accuracy			0.92	20000
macro avg	0.92	0.93	0.92	20000
weighted avg	0.93	0.92	0.92	20000

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_estimator(
    rf,
    X_test,
    y_test
)
plt.show()
```



```
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf.feature_importances_
})

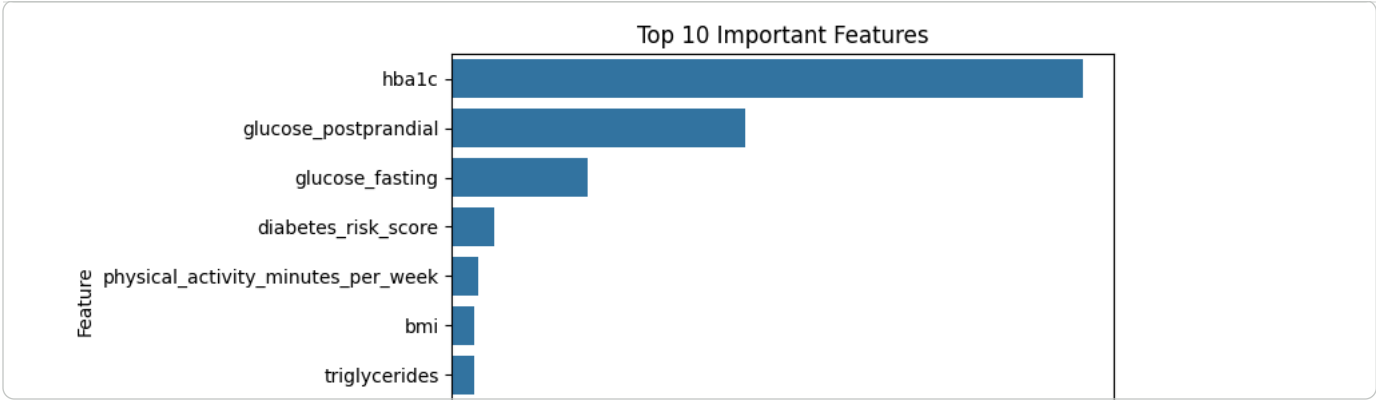
feature_importance.sort_values(
    by='Importance',
    ascending=False
).head(10)
```

	Feature	Importance
21	hba1c	0.422707
19	glucose_postprandial	0.196395
18	glucose_fasting	0.091287
22	diabetes_risk_score	0.028280
2	physical_activity_minutes_per_week	0.017163
9	bmi	0.015213
17	triglycerides	0.015072
20	insulin_level	0.014763
0	age	0.014701
14	cholesterol_total	0.014461

```
top_features = feature_importance.sort_values(
    by='Importance',
    ascending=False
).head(10)

sns.barplot(
    x='Importance',
    y='Feature',
    data=top_features
)

plt.title('Top 10 Important Features')
plt.show()
```



Conclusion

The "Diabetes Prediction Using Machine Learning" project was developed with the primary objective of predicting whether a patient is diagnosed with diabetes using demographic, lifestyle, and clinical health data. Diabetes is one of the fastest-growing chronic diseases